Sail-RISCV-WASM: A Browser-Native RISC-V Toolchain and Debugging Workbench

Abstract

This paper presents Sail-RISCV-WASM, which addresses three common limitations of existing browser-based RISC-V tools: fragmented capabilities, limited configurability, and disconnected build/debug pipelines. The system uses `sail-riscv` as its semantic baseline and compiles it to WebAssembly, forming a three-layer architecture in a pure browser environment: a Sail decode/execute layer, a toolchain layer (gas/ld/objdump), and a metadata layer based on the RISC-V UDB. Based on this architecture, the paper defines two core workflows. The first is configuration-sensitive online encode/decode with instruction metadata navigation for cross-configuration behavior comparison. The second is an in-browser assemble-to-ELF, execute, and interactive debugging loop, supporting instruction-level stepping, source-line stepping, synchronized source/disassembly views, and register/memory tracing. Results show that the system provides a complete single-page flow from exploration to build to diagnosis, with strong extension coverage and configuration flexibility.

Introduction

Browser-native delivery lowers the barrier to ISA tooling, but many browser tools still focus on only one stage, such as classroom visualization or standalone conversion. This leads to fragmented workflows, limited extension configurability, and weak linkage between source, execution, and diagnosis. Sail-RISCV-WASM addresses this gap with a single browser workspace for configuration-sensitive exploration, in-browser build, execution, and interactive debugging. It is grounded in the RISC-V ISA specifications[1] and sail-riscv[2], with broad extension coverage and fine-grained config- uration control. The practical motivation is to reduce context switches across tools. Instead of editing source in one environment, rebuilding in another, and diagnosing traces in a third, the workflow remains in one browser page with a shared project state. This shortens it- eration cycles for teaching, issue reproduction, and ISA-level debugging tasks where quick configuration changes and immediate feedback are more important than full system emulation.

Contributions This work contributes an end-to- end browser pipeline with no server-side execution plane, Sail-grounded decode and execute paths, broad extension coverage with fine-grained configuration switches, an in-browser source-to-ELF flow via gas/ld, and synchronized instruction-step/source-line-step de- bugging with register and memory traces.

Methodologies

The implementation combines three browser-native layers: a Sail execution/decode layer compiled to WebAssembly, an in-browser gas/ld/readelf/objdump toolchain, and an instruction metadata layer indexed from RISC-V UDB content.

At the interface layer, the runtime editor exposes three primary editable artifacts: the Sail configuration JSON, the RISC-V assembly source, and the linker script. These files are edited directly in-browser and passed to the worker pipeline without server-side preprocessing. This design keeps configuration intent and generated execution artifacts tightly coupled, which helps users reason about why decode behavior, linking outcome, or runtime state changed after an edit.

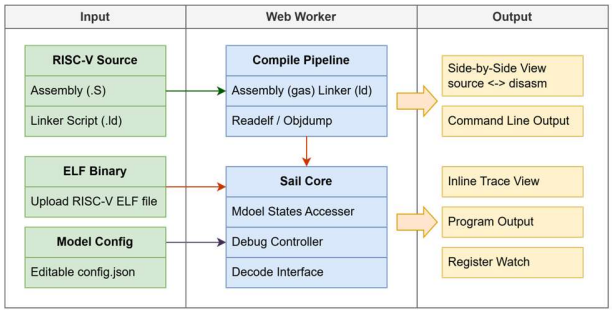

Figure 1: Architecture

Model and encode/decode Runtime decode and execution are authoritative in Sail. Decoding reads user-provided JSON configuration (for example XLEN and extension enablement), so the same machine encoding can decode differently under different configurations. Explorer-side encoding is intentionally exploratory: it is metadata-backed because Sail does not expose a stable browser-ready assembler interface for arbitrary instruction text. The encoder is XLEN-aware but does not enforce full extension legality; final legality is determined only by Sail decode and execute.

Debug mapping and trace path Before debugging, the worker assembles and links source files into a debug-capable ELF, then extracts address-to-source-line and address-to-disassembly-line mappings. Source-line stepping uses nearest-address association of these mappings. This strategy is best-effort; failure modes include stripped DWARF, optimization-driven aliasing, and repeated macro ranges. When mapping is weak, the UI falls back to disassembly anchoring and then PC anchoring. Build and debug operations run in a dedicated worker.

WebWorker and browser debugging interfaces The system separates heavy computation from interactive control. The worker executes assembly, linking, disassembly, line mapping, and log aggregation. During runtime control, it calls browser-oriented debugging interfaces added around sail-riscv (session initialization, instruction-step, source-line-step, run, state readout, and reset). The frontend triggers these capabilities through a unified RPC surface and consumes structured debug-state updates.

Workflows

Online Encode/Decode and Metadata Navigation The exploration workflow accepts binary, hex, or assembly input and normalizes it into a shared instruction state. After configuration changes, it re-runs decode, refreshes bit-field/operand highlights, and updates extension tags plus documentation-oriented metadata. This supports quick comparison of one encoding under different configuration presets.

The same page also serves as a lightweight instruction reference surface. Users can inspect extension tags and metadata fields while iterating on encodings, which makes it easier to connect decode outcomes with documented instruction structure and extension scope. This is especially useful when debugging legality changes caused by toggling architecture options in the configuration file.

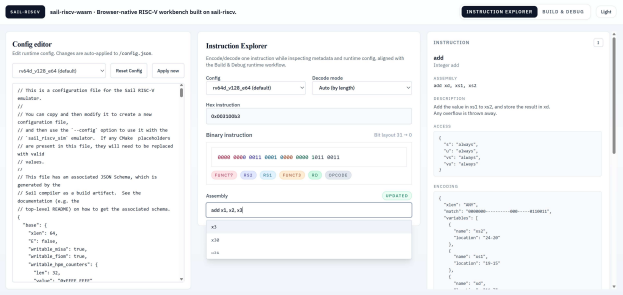

Figure 2: Encode/decode and metadata page.

Build, Execute, and Debug The runtime workflow supports editing assembly and linker scripts, building ELF binaries in-browser, and executing them in instruction-step or source-line-step mode. It also supports direct ELF upload. During stepping, the editor keeps source and disassembly synchronized and highlights mapped regions, while register and memory events appear in logs and inline annotations. This closes the loop from source editing to diagnosis in one page.

The runtime page includes terminal-style output panes that aggregate build logs, Sail trace lines, and execution status messages in real time. This output model gives users both high-level progress feedback (build succeeded, initialized, halted) and low-level diagnostic evidence (register writes, memory writes, and committed instruction context) without leaving the debugging screen. In practice, this reduces friction when diagnosing short bare-metal programs that rely on HTIF-style I/O and explicit memory-mapped interactions.

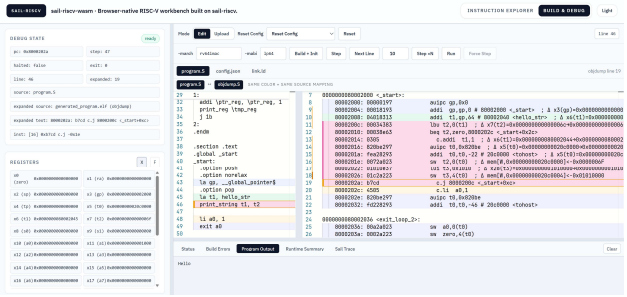

Figure 3: Compile-execute-debug page.

Discussion

We summarize scope differences across three browser-facing tools: CPUlator[3], WebRISC-V[4] and Sail-RISCV-WASM. The comparison focuses on deployment model, extension coverage, trace visibility, and configuration strength.

Table 1: Tool comparison.

Capability	CPUlator	WebRISC-V	This work
No server	No	No	Yes
Extension support	Partial	Partial	Yes
Reg/Mem trace	Partial	Partial	Yes
Configurability	No	No	Yes

Summary Sail-RISCV-WASM provides an accessible and engineering-oriented RISC-V workflow: browser-native deployment, configuration-sensitive decode exploration, in-browser assembly and linking, synchronized source/disassembly stepping, and trace-driven diagnosis in one environment. Compared with common browser-only flows, it combines broader extension coverage and stronger configuration control in the same loop. The current scope is single-core bare-metal execution; future work includes broader configuration coverage and stronger differential checks against desktop references.

References

[1] Andrew Waterman and Krste Asanovic. The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA. url: https://docs.riscv.org/reference/isa/_attachments/riscv-unprivileged.pdf

[2] Alasdair Armstrong sail-riscv: Formal Specification of the RISC-V ISA. url: https://github.com/riscv/sail-riscv

[3] CPUlator Computer System Sim-ulator. 2026. url: https://cpulator.01xz.net/

[4] Mariotti, G., & Giorgi, R. (2022). WebRISC-V: A 32/64-bit RISC-V pipeline simulation tool. *SoftwareX, 18, 1–7.*